Gabriel Calvo Catalan | Lucas Diaz Lopez

1) Informació sobre ús d'IA:

Contesta a les següents preguntes:

(En totes les preguntes pots no marcar cap opció o marcar-ne més d'una)

1) Explica com has aplicat la IA per fer la pràctica.

a) La IA m'ha ajudat a entendre conceptes tècnics que apareixen a l'enunciat

h) La IA m'ha ajudat a polir l'informe

f) La IA m'ha ajudat a provar el codi

2) Quines eines has fet servir? Si n'has fet servir més d'una índica per a què has fet servir cadascuna.

a) Gemini

3) En general quin ordre has fet servir (tant per la generació de codi com per la redacció de l'informe)?

b) He començat fent jo una primera proposta i després la IA ho ha completat/polit.

4) En la part de generació/revisió de codi quin mètode has fet servir?

a) He treballat amb l'eina de xat habitual (copiant/enganxant codi del xat als meus fitxers).

5) Has fet servir agents? Més d'un simultàniament? Indica quines eines i com ho has fet.

no hem fet servir agents

6) Tal com has fet servir la IA, quant de temps (en %) penses que t'has estalviat fer la pràctica (per exemple un 20% si sense IA eren 50 minuts i amb IA 40 minuts)?

un 15/20%, ja que va ser només per entendre conceptes nous

7) Tal com has fet servir la IA, penses que has pogut millorar la qualitat de les teves respostes (p. ex., més complets, més detallats, ...) a l'enunciat de la pràctica que sense IA?

No, ja que no ha afectat les respostes

8) Quin impacte penses que té la IA sobre el teu aprenentatge:

a) Veig un impacte bàsicament positiu, aprenc més de manera més eficient.

9) Segons la teva experiència, quin ús de la IA recomanaries per fer aquesta pràctica?

Per entendre conceptes de la practica que no s'entenen a primeres

Gabriel Calvo Catalan | Lucas Diaz Lopez

2) Explicació tècnica del desenvolupament de la pràctica

2.1 Entorn i configuració

El desenvolupament de la pràctica s'ha dut a terme sobre un entorn basat en Node.js utilitzant el framework Express per a la construcció del servidor HTTP de backend. L'arquitectura es divideix en un conjunt d'endpoints RESTful que gestionen les operacions CRUD sobre les dades de lloguer (rentals). Per a la persistència de dades, s'ha utilitzat un fitxer en format JSON directament integrat mitjançant el mòdul natiu fs de Node.js, la qual cosa permet una manipulació directa i sense la necessitat de desplegar una base de dades relacional o NoSQL complexa.

Els components tècnics i les tecnologies principals emprades són:
- Node.js: entorn d'execució en temps real que permet executar codi JavaScript al costat del servidor de manera asíncrona i orientada a esdeveniments.
- Express.js: framework de servidor minimalista encarregat d'encaminar les peticions HTTP (GET, POST), gestionar middlewares i servir els fitxers estàtics de la interfície d'usuari.
- Frontend estàtic (HTML/CSS/JS): ubicat a la carpeta frontend, es comunica amb la interfície API mitjançant crides asíncrones utilitzant l'API Fetch nactiva dels navegadors.
- Mòdul fs (File System): utilitzat per realitzar opreacions d'E/S asíncrones per llegir i escriure el fitxer JSON que actua com a magatzem persistent.
- jsonwebtoken (JWT): llibreria emprada per a l'emissió i verificació de tokens d'autenticació signats per protegir determinades rutes del servidor.
- Docker i Docker Compose: eines de contenidorització utilitzades per empaquetar l'aplicació juntament amb les seves dependències, garantint la reproductibilitat en qualsevol entorn d'execució.

2.2 Solució triada

La solució dissenyada s'estructura al voltant d'una API REST modular i una interfície web senzilla però funcional. Els serveis principals de l'API inclouen:
- POST /newrental: permet el registre d'una nova comanda de lloguer validant els camps rebuts (tipus de motor, qualificació de CO2, dies, unitats i descompte).
- GET /rentals: retorna la llista completa de registres emmagatzemats en format JSON.
- POST /login: gestiona la validació de credencials i retorna un token JWT per autenticar l'usuari.
- Protecció de rutes: ús de middleware personalitzat per verificar la validesa del token JWT en les capçaleres d'autorització.
- Contenidorització Docker: construcció d'una imatge basada en Node.js lleugera per facilitar el desplegament immediat sense configuració prèvia a la màquina hoste.
El flux de treball complet segueix el patró client-servidor: quan un usuari interacciona amb el formulari web o envia una petició directe des de la línia de comandes, el servidor Express rep les dades, les processa, actualitza el fitxer JSON si cal i respon amb els codis d'estat HTTP corresponents (com 200 OK o 201 Created). Aquesta arquitectura s'ha seleccionat per complir amb la simplicitat de la pràctica mantenint un bon nivell de desacoblament.

La lògica bàsica és: el client envia JSON, el servidor l'interpreta, l'emmagatzema i respon amb codi HTTP adequat. La persistència es fa en un fitxer JSON perquè l'enunciat ho demana i perquè és una solució simple i adequada per a aquesta pràctica.

2.3 Proves realitzades

Per tal de verificar el correcte funcionament del sistema, s'han dut a terme dos tipus de proves: mitjançant la interfície web i mitjançant l'eina de línia de comandes curl. Les comprovacions clau han inclòs:
- Creació de nous lloguers tant des del formulari com mitjançant peticions POST amb JSON.
- Consulta de la llista de rentals actualitzada en temps real des de la vista de frontend i des del terminal.
- Generació i emissió de tokens d'autenticació JWT mitjançant la petició POST a /login amb les credencials correctes.
- Validació de la protecció d'endpoints privats comprovant el rebut de respostes d'error en cas de no incloure la capçalera Bearer Token.

2.4 Avaluació de la solució

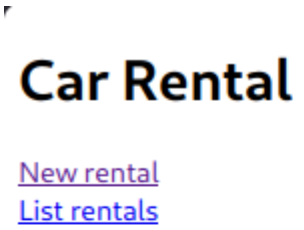
La solució implementada s'ajusta plenament als requisits funcional de la pràctica. L'ús d'Express combinat amb la interfície web garanteix una bona usabilitat, mentre que la implementació de JWT aporta una capade seguretat en les comunicacions client-servidor. Així mateix, la inclusió de Docker garanteix l'aïllament de l'entorn d'execució. Com a limitació tècnica, cal destacar que la persistència basada en fitxers JSON pot presentar problemes de concurrència i escalabilitat en entorns de producció reals, on seria recomanable migrar cap a una base de dades relacional o documental.

3) Captures requerides

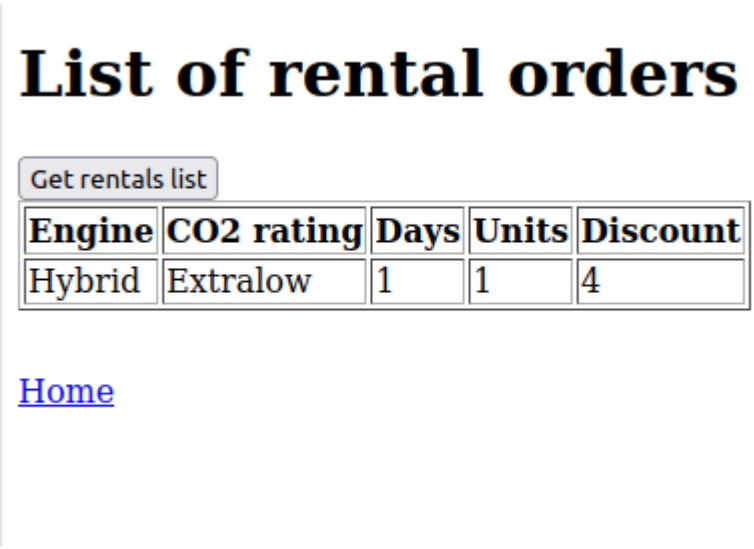
Captura 1: Execució de peticions curl des del terminal per crear un nou lloguer (POST /newrental) i verificar la llista actualitzada (GET /rentals).



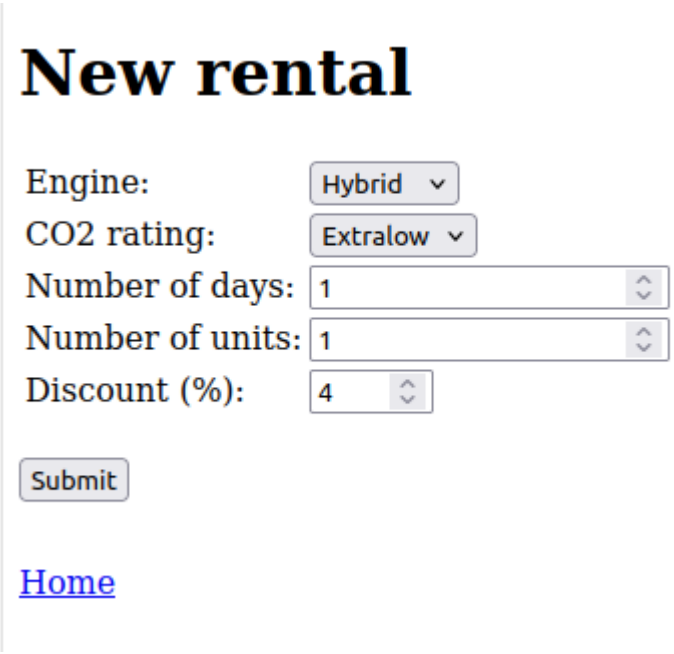
Captura 2: Pàgina principal de l'aplicació web ("Car Rental") amb els enllaços d'accés ràpid al formulari de nou lloguer i a la llista de lloguers.



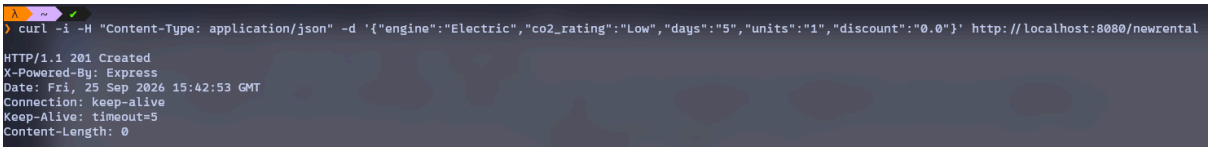
Captura 3: Vista de la llista de comandes de lloguer ("List of rental orders") mostrades en una taula HTML des del frontend.



Captura 4: Formulari web per introduir les dades d'un nou lloguer ("New rental"), incloent el tipus de motor, emissions, dies, unitats i descompte.



Captura 5: Petició POST a /newrental enviant un JSON amb les dades del lloguer i obtenint una resposta de confirmació amb codi HTTP 201 Created.



Captura 6: Resposta JSON obtinguda mitjançant una petició GET a /rentals que mostra tots els registres emmagatzemats al servidor.



Captura 7: Generació d'un token d'autenticació JWT realitzant una petició POST al punt d'entrada /login amb credencials d'usuari.



4) Observació final

La pràctica s'ha resolt amb una arquitectura de servidor Express, dades persistents en JSON, interfície web i autenticació JWT addicional. La dockerització permet executar l'aplicació de manera independent i reproduïble en qualsevol màquina, i la web fa que el sistema sigui més usable i comprensible per a un usuari final.